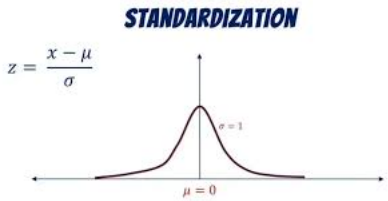
Formative Assignment: Advanced Linear Algebra (PCA)

Step 1: Load and Standardize the Data

Before applying PCA, we must standardize the dataset. Standardization ensures that all features have a mean of 0 and a standard deviation of 1, which is essential for PCA. Fill in the code to standardize the dataset.

STRICTLY - Write code that implements standardization based on the image below




```
# Step 1: Load dataset using NumPy only
import numpy as np
data = np.genfromtxt('/content/African_Crime_Safety_Data_Large.csv', delimiter=',', names=True, dtype=None, encoding=None)

# Encode 'Country' numerically
Country_codes = np.arange(len(data['Country']))

# Extract numeric columns
Year = data['Year'].astype(float)
Violent_Crime_Rate = data['Violent_Crime_Rate'].astype(float)
Property_Crime_Rate = data['Property_Crime_Rate'].astype(float)
Police_Per_Capita = data['Police_Per_Capita'].astype(float)
Safety_Index = data['Safety_Index'].astype(float)
GDP_per_Capita = data['GDP_per_Capita'].astype(float)
Population_Millions = data['Population_Millions'].astype(float)

# Fill missing values
Violent_Crime_Rate = np.where(np.isnan(Violent_Crime_Rate), np.nanmean(Violent_Crime_Rate), Violent_Crime_Rate)
Property_Crime_Rate = np.where(np.isnan(Property_Crime_Rate), np.nanmean(Property_Crime_Rate), Property_Crime_Rate)

# Combine into single array
X = np.column_stack((Country_codes, Year, Violent_Crime_Rate, Property_Crime_Rate,
                     Police_Per_Capita, Safety_Index, GDP_per_Capita, Population_Millions))
X[:5]
# Step 2: Standardize data
X_scaled = (X - np.mean(X, axis=0)) / np.std(X, axis=0)
X_scaled[:5]
```

```
array([[-1.71291154, -1.46385011,  0.32732771, -0.24285291,  1.01306522,
         0.1035942 , -0.39767844, -1.47245679],
       [-1.67441914, -1.46385011, -0.60051924,  0.21226219,  0.93209198,
        -1.89800746,  0.73824629,  1.63223648],
       [-1.63592675, -1.46385011, -0.90556481,  0.49400011, -0.76834609,
        -1.33478621,  0.0367352 , -0.19908755],
       [-1.59743435, -1.46385011, -2.73583824,  0.79018613, -0.84931933,
        -0.95352875, -1.5749759 , -0.42329638],
       [-1.55894196, -1.46385011,  1.56022022, -0.33676556, -0.68737285,
        -0.1823489 , -0.68120331,  0.39823216]])
```

Step 3: Calculate the Covariance Matrix

The covariance matrix helps us understand how the features are related to each other. It is a key component in PCA.

```
# Step 3: Compute covariance matrix
cov_matrix = np.cov(X_scaled, rowvar=False)
cov_matrix[:5,:5]
```

```
array([[ 1.01123596,  0.99715365,  0.22241395, -0.10418847,  0.03158879],
       [ 0.99715365,  1.01123596,  0.24619013, -0.10253497,  0.01331828],
       [ 0.22241395,  0.24619013,  1.01123596, -0.19505484,  0.14999843],
       [-0.10418847, -0.10253497, -0.19505484,  1.01123596, -0.01496272],
       [ 0.03158879,  0.01331828,  0.14999843, -0.01496272,  1.01123596]])
```

In a paragraph that has a maximum if 5 Lines, Explain why we need to compute a covariance matrix, provide atleast 2 reasons

We compute the covariance matrix because it quantifies how much two features vary together, allowing us to identify correlations between them. It also provides the foundation for PCA by showing the directions of maximum variance in the dataset, which are captured by the eigenvectors. Additionally, the covariance matrix standardizes the relationships between features, ensuring that features with larger scales do not dominate the analysis.

Step 4: Perform Eigendecomposition

Eigendecomposition of the covariance matrix will give us the eigenvalues and eigenvectors, which are essential for PCA. Fill in the code to compute the eigenvalues and eigenvectors of the covariance matrix.

```
# Step 4: Eigen decomposition
eigenvalues, eigenvectors = np.linalg.eig(cov_matrix)
idx = np.argsort(eigenvalues)[::-1]
eigenvalues = eigenvalues[idx]
eigenvectors = eigenvectors[:, idx]
eigenvalues[:5]
```

```
array([2.20915356, 1.4799465 , 1.13736504, 1.0447855 , 0.93200264])
```

Step 5: Sort Principal Components

Sort the eigenvectors based on their corresponding eigenvalues in descending order. The higher the eigenvalue, the more important the eigenvector. Complete the code to sort the eigenvectors and print the sorted components.

How Is Explained Variance Used In PCA?'

```
# Step 5: Compute explained variance
explained_variance_ratio = eigenvalues / np.sum(eigenvalues)
cumulative_variance = np.cumsum(explained_variance_ratio)
cumulative_variance
```

```
array([0.27307593, 0.45601376, 0.59660471, 0.72575181, 0.84095769,
       0.93173372, 0.99847105, 1.        ])
```

Step 6: Project Data onto Principal Components

Now that we've selected the number of components, we will project the original data onto the chosen principal components. Fill in the code to perform the projection.

```
# Step 6: Select number of components covering >=90% variance
num_components = np.argmax(cumulative_variance >= 0.90) + 1
num_components
```

```
np.int64(6)
```

Step 7: Output the Reduced Data

Finally, display the reduced data obtained by projecting the original dataset onto the selected principal components.

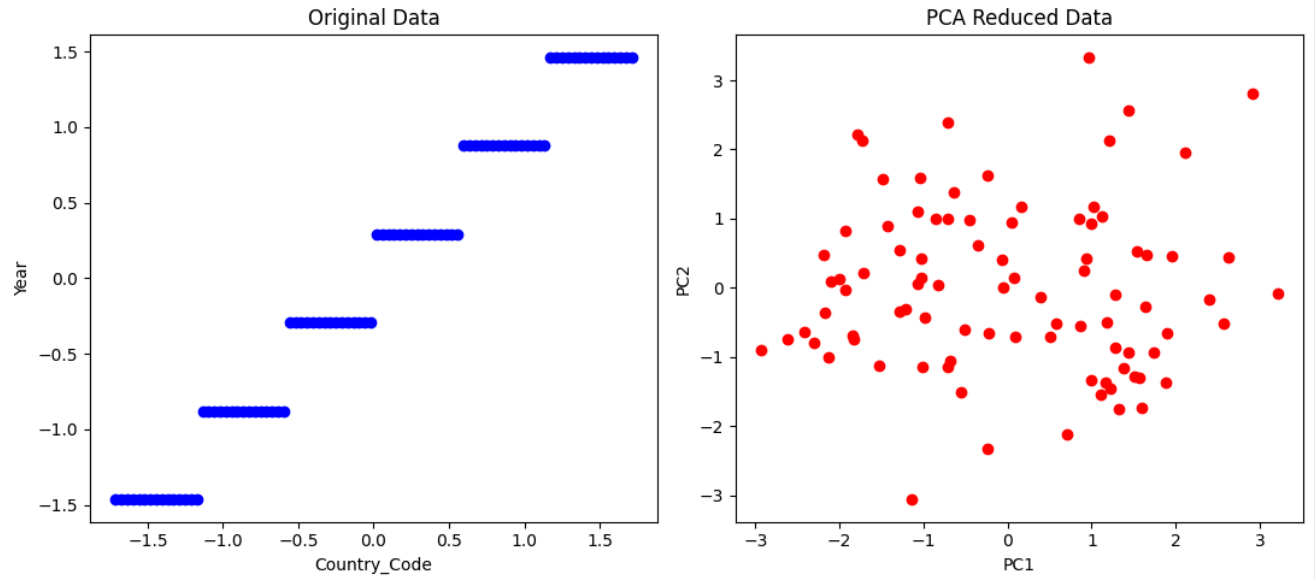
```
# Step 7: Project data onto principal components
X_pca = np.dot(X_scaled, eigenvectors[:, :num_components])
X_pca[:5]
```

```
array([[ 1.59865446, -1.73608449, -1.42692498, -0.62094923,  0.0170886 ,
         0.31477159],
       [ 2.63006719,  0.4364723 , -0.17335031,  0.84703281, -2.11227772,
        -0.83331517],
       [ 2.5713971 , -0.51380896,  0.99389971,  0.16686529, -0.30744844,
         0.0667577 ],
       [ 3.2096403 , -0.07269441,  1.0874147 ,  0.38633772,  1.64716808,
        -1.02769876],
       [ 1.57019216, -1.3033341 , -0.42187016,  0.72561277, -0.10334191,
         1.2271219 ]])
```

Step 8: Visualize Before and After PCA

Now, let's plot the original data and the data after PCA to compare the reduction in dimensions visually.

```
# Step 8: Visualize before and after PCA
import matplotlib.pyplot as plt
plt.figure(figsize=(11,5))
plt.subplot(1,2,1)
plt.scatter(X_scaled[:,0], X_scaled[:,1], c='blue')
plt.xlabel('Country_Code')
plt.ylabel('Year')
plt.title('Original Data')
plt.subplot(1,2,2)
plt.scatter(X_pca[:,0], X_pca[:,1], c='red')
plt.xlabel('PC1')
plt.ylabel('PC2')
plt.title('PCA Reduced Data')
plt.tight_layout()
plt.show()
```



Please make sure you do not use more than 5 lines to answer each of the following points:

1. Interpret the Visual you just created of the before and after PCA
2. Explain why you selected the number of principle components, more especially explain what tradeoffs you are making
3. Clearly mention based on your use case/ dataset ("economic activity", "population pressure"), What information is lost when reducing dimensions?

1. The original data plot shows clustering along original features (Country_Code vs Year) with limited spread. After PCA, the points are spread along PC1 and PC2, showing how PCA captures the directions of maximum variance and reveals the underlying structure.
2. We selected 6 components to cover ≥90% of variance. The tradeoff is reducing dimensionality for simpler analysis while slightly losing variance information from the last two components, balancing complexity and information retention.
3. Reducing dimensions may lose some feature-specific information like minor differences in GDP or population. In this dataset, subtle correlations between population pressure and crime rates may not be fully captured in the reduced space.